Databases and Analytics Assignment

Module: Database and Analytics

Module Code: CP60056E

School: School of Computing and Engineering

Module Leader: Dr. Shidrokh Goudarzi

Submission Date: 12 May 2026

-

Student Name: Mohammed Adnan Osman

Student Number: 33114153

Report Word Count: 4362

-

GitHub Repository: https://github.com/adnan-1055/Databases-and-Analytics

Contents

Introduction 4
SQL in R 4
SQL Syntax and Filtering 5
Query 1: Failed deliveries by zone 5
Query 2: Complaints by service type 6
Data Retrieval and Manipulation 7
Query 3: Driver route overrides 7
Query 4: Complaint resolution time by severity 8
Query Optimization 9
Query 5: High value failed orders 9
Relational Database Concepts 10
Interpretation of Results 10
R Analytics 11
Statistical Analysis 11
Summary Statistics 11
Data Manipulation 12
Data Visualization 13
Chart 1: Delivery status breakdown 13
Chart 2: Complaints by severity 15
Chart 3: Customer ratings after delivery 17
Chart 4: Customer ratings by delivery status 19
Interpretation of Outputs 20
Python Data Processing 20
Data Cleaning and Manipulation 21
Missing values check 21
Data Analysis with Pandas and NumPy 22
NumPy statistics 22
Data Visualization 23

Chart 1: Delivery status counts 23
Chart 2: Fuel cost by delivery status 24
Chart 3: Complaint types 25
Chart 4: Correlation heatmap 27
Presentation of Results 28
MongoDB Development 28
Database Creation and Configuration 28
NoSQL Data Modelling 29
JSON/BSON Format Handling 30
CRUD Operations using PyMongo 30
CREATE: inserting data 30
READ: querying documents 31
UPDATE: escalating complaints 31
DELETE: removing invalid records 32
Aggregation pipeline 32
Database Design for NorthStar 33
Query Optimization 33
Indexing Strategy 33
Query Performance Analysis using Explain Plans 34
Before indexing 34
After Indexing 34
Optimization Techniques 35
Index information 35
Justification and Evaluation 36
Conclusion 36
GitHub Repository 37
References 37

Introduction

NorthStar Urban Mobility and Logistics is a large regional organisation operating across several major UK cities. Over six years of rapid expansion through mergers, the company now manages shuttle services, last-mile delivery, warehouse dispatch, EV charging hubs, and a mobile customer platform. Despite rising demand, the business has become increasingly inefficient with rising complaints, missed delivery windows, and operational costs growing faster than revenue, delivery windows are missing, and operational costs have grown faster than revenue.

The core problem is that NorthStar's data environment has outgrown its ability to manage it. Structured operational systems record bookings, journeys and payments, while newer services generate complex semi-structured data including app interactions, driver event streams and complaint histories that do not fit neatly into existing relational structures.

This report presents an integrated analytical solution using SQL in R, R analytics, Python and MongoDB Atlas to identify inefficiencies, uncover patterns and design a NoSQL database capable of handling NorthStar's complex and evolving data needs.

SQL in R

The sqldf package was used within R to run SQL queries directly on the NorthStar dataframes. This approach allows structured data to be queried efficiently without requiring a separate database system. Five queries were written to investigate key operational problems identified in the case study.

SQL Syntax and Filtering

Query 1: Failed deliveries by zone

	pickup_zone	failed_deliveries
1	RiverSide	14
2	EAST	11
3	Ctr	11
4	Central	11
5	CENTRAL	11
6	South	10
7	north	8
8	East	8
9	Airport	8
10	West	7
11	WEST	7
12	North	7
13	NORTH	7
14	SOUTH	4
15	Riverside	4
16	AIRPORT	4

Riverside recorded the highest number of failed deliveries with 14, followed by EAST, Central and CENTRAL, each with 11. Inconsistent naming conventions such as Central, CENTRAL and Ctr suggest the same zone is recorded differently across the system, indicating a data quality issue that undermines reliable reporting across the zone network. If the Central zone variants are combined, the true failure count for that area exceeds Riverside, making it the most problematic zone in the network. NorthStar should implement a standardized zone naming convention enforced at the point of data entry to ensure accurate zone-level reporting going forward.

-

```
failed_by_zone <- sqldf("

SELECT o.pickup_zone,

  COUNT(*) AS failed_deliveries

FROM deliveries d

JOIN orders o ON d.order_id = o.order_id

WHERE d.delivery_status = 'Failed'

GROUP BY o.pickup_zone

ORDER BY failed_deliveries DESC
```

")

print(failed_by_zone)

-

The sqldf package runs this query directly on the NorthStar dataframes within R. A JOIN combines the deliveries and orders tables on the shared order_id field. A WHERE clause filters for failed deliveries only, GROUP BY aggregates the count by zone, and ORDER BY ranks zones from highest to lowest failure rate. Selecting only the required columns rather than using SELECT * reduces the memory usage and thus improves the query performance.

Query 2: Complaints by service type

	service_type	total_complaints
1	Passenger	84
2	Retail	83
3	Parcel	77
4	Business	39
5	Medical	37

Passenger, Retail and Parcel services generated the highest complaint volumes with 84, 83 and 77 incidents. No single service type is performing well, suggesting the problem is systemic rather than confined to one area and pointing to shared issues in dispatch or resource allocation that affect all service lines. The narrow gap between the top three service types is significant, it rules out the possibility that poor performance is concentrated in one area and confirms that the underlying causes are shared across the entire operation. Business and Medical services, while lower in volume at 39 and 37, represent NorthStar's highest value contracts and even a small number of complaints in these categories carries disproportionate financial and reputational risk.

-

```
complaints_by_service <- sqldf("

SELECT o.service_type,

  COUNT(*) AS total_complaints

FROM complaints c

JOIN orders o ON c.order_id = o.order_id

GROUP BY o.service_type

ORDER BY total_complaints DESC
```

")

print(complaints_by_service)

-

An inner join between the complaints and orders datasets on order_id maps each complaint to its corresponding service type. COUNT(*) totals the complaint volume per category and GROUP BY partitions the results by service type. ORDER BY ranks service lines from highest to lowest complaint volume, making it easier to find where the failures are located in the business.

Data Retrieval and Manipulation

Query 3: Driver route overrides

	driver_id	total_overrides
1	D127	17
2	D130	16
3	D087	16
4	D131	15
5	D108	15
6	D105	14
7	D069	14
8	D028	13
9	D017	13
10	D104	12

Driver D127 recorded the highest number of manual route overrides at 17. High override counts suggest poor route planning by the system or drivers deliberately avoiding performance targets, which aligns directly with concerns raised by NorthStar's operations director. The fact that all ten drivers in the top list record at least 12 overrides suggest this is not an isolated behavioral issue but a widespread operational problem affecting the entire driver's workforce. NorthStar should cross-reference high override drivers with their delivery failure rates and customer ratings to determine whether the root cause lies with the routing algorithm or with individual driver behavior and introduce override monitoring policies accordingly.

-

```
driver_overrides <- sqldf("

SELECT driver_id,

  SUM(manual_route_override_count) AS total_overrides
```

```
FROM deliveries

GROUP BY driver_id

ORDER BY total_overrides DESC

LIMIT 10

")

print(driver_overrides)
```

-

SUM aggregates the total manual override count per driver across all their deliveries and GROUP BY partitions results by driver_id. ORDER BY ranks drivers by override volume from highest to lowest, isolating the individuals with the most frequent route deviations. LIMIT 10 restricts the output to the top ten results, reducing unnecessary processing and focusing the analysis on the main contributors to the problem.

Query 4: Complaint resolution time by severity

	severity	avg_resolution_days
1	High	13.116883
2	Low	6.563380
3	Medium	6.168605

High severity complaints take an average of 13.1 days to resolve, more than double the time taken for Medium and Low severity cases. This indicates a significant failure in NorthStar's customer service processes, particularly for the most serious complaints. A resolution time of 13.1 days for the most serious complaints is the inverse of what an effective complaints management system should deliver, high severity cases should be triaged immediately and resolved quickly, not left outstanding for nearly two working weeks. NorthStar should introduce tiered service level agreements with strict resolution time targets for each severity level and implement automated escalation rules to prevent high severity complaints from being deprioritized within the general complaints queue.

-

```
resolution_time <- sqldf("

SELECT severity,

  AVG(resolution_days) AS avg_resolution_days

FROM complaints
```

```
GROUP BY severity

ORDER BY avg_resolution_days DESC

")

print(resolution_time)
```

-

AVG calculates the mean resolution time per severity group and GROUP BY partitions the data by severity level. ORDER BY sorts the results from slowest to fastest resolution, showing the performance gap between High severity complaints and the rest. This allows to show the data in a comparable summary metric, which provides evidence to support a review of the business current service level agreements.

Query Optimization

Query 5: High value failed orders

	order_id	service_type	order_value	delivery_status
1	O01207	Business	321.68	Failed
2	O01112	Business	278.14	Failed
3	O00173	Passenger	249.32	Failed
4	O00919	Retail	229.93	Failed
5	O01143	Retail	217.11	Failed
6	O00585	Medical	215.96	Failed
7	O00035	Business	209.07	Failed
8	O00081	Passenger	208.38	Failed
9	O00536	Medical	204.12	Failed
10	O00921	Retail	198.22	Failed

The top failed order was a business contract worth £321.68. Multiple high value Business, Passenger and Retail orders appear in the failed list, confirming the finance director's concern that delivery failures are directly costing NorthStar revenue on its highest value contracts. The ten highest value failed orders alone represent a combined revenue loss of over £2,300, and this figure only captures the lost income of the one. It does not account for the full operational cost of each failed attempt including driver time, fuel and vehicle usage that was expended with no return. NorthStar should introduce priority handling protocols for orders above £100 in value to ensure they receive additional oversight before and during dispatch, directly protecting the highest value contracts from the delivery failures that are currently damaging both revenue and client relationships.

-

```
high_value_failed <- sqldf("
SELECT o.order_id,
  o.service_type,
  o.order_value,
  d.delivery_status
FROM orders o
JOIN deliveries d ON o.order_id = d.order_id
WHERE d.delivery_status = 'Failed'
AND o.order_value > 100
ORDER BY o.order_value DESC
LIMIT 10
")
print(high_value_failed)
```

-

A JOIN links the orders and deliveries tables on order_id and a WHERE clause filters to failed deliveries with an order value above £100. ORDER BY ranks results by order value descending and LIMIT 10 restricts the output to the ten highest value losses. Selecting only the four required fields rather than using SELECT * reduces data retrieval and keeps the query focused on the most financially significant failures.

Relational Database Concepts

The queries I used demonstrate the main relational database concepts. JOIN operations combine the orders and deliveries tables on the shared order_id field, reflecting a one-to-many relationship between orders and their delivery records. GROUP BY aggregates data across multiple records and WHERE clauses filter results to specific conditions. These techniques demonstrate how relational data can be structured and queried efficiently within R.

Interpretation of Results

Across all five queries, a consistent pattern of operational failure emerges. RiversSide zone has the highest delivery failure rate, Passenger and Retail services generate the most

complaints, and high value orders are being lost to delivery failures. Manual route overrides occur at nearly one per driver and high severity complaints are taking over 13 days to resolve. These findings directly support the concerns raised by NorthStar's operations director, customer experience director and finance director, and provide a data backed recommendations made in the conclusion.

R Analytics

The ggplot2 package was used within R to perform statistical analysis and create visualisations of the NorthStar dataset. A summary statistics table was produced to understand the distribution of key operational metrics before charting.

Statistical Analysis

Summary Statistics

```
 route_distance_km manual_route_override_count customer_rating_post_delivery
 Min.   : 1.200    Min.   :0.0000              Min.   :1.000
 1st Qu.: 9.135    1st Qu.:0.0000              1st Qu.:3.360
 Median :12.840    Median :1.0000              Median :4.040
 Mean   :13.909    Mean   :0.9695              Mean   :3.865
 3rd Qu.:16.835    3rd Qu.:2.0000              3rd Qu.:4.550
 Max.   :41.940    Max.   :7.0000              Max.   :5.000
                                               NA's   :14
 fuel_or_charge_cost
 Min.   : 2.500
 1st Qu.: 9.925
 Median :12.645
 Mean   :12.841
 3rd Qu.:15.697
 Max.   :29.430
```

-

summary(deliveries[, c("route_distance_km",

"manual_route_override_count",

"customer_rating_post_delivery",

"fuel_or_charge_cost")])

-

The summary() function was applied to four key numeric columns in the deliveries dataset to produce descriptive statistics including minimum, maximum, mean, median and

quartile values. The summary statistics reveal that the average route distance is 13.9km with a maximum of 41.9km, suggesting some routes are unusually long. The average manual override count is nearly 1 per delivery, and 14 customer ratings are missing. This is a data quality issue that should be addressed. The median customer rating of 4.04 suggests that while overall satisfaction is decent, the mean of 3.865 is pulled down by a significant number of low ratings from failed and delayed deliveries.

Data Manipulation

Before visualization, the deliveries dataset was loaded into R and inspected using the str() function to confirm column types and data structure. The summary() function was applied to key numeric columns to produce descriptive statistics including minimum, maximum, mean, median and quartile values. This allowed patterns and anomalies in the data to be identified before charting.

-

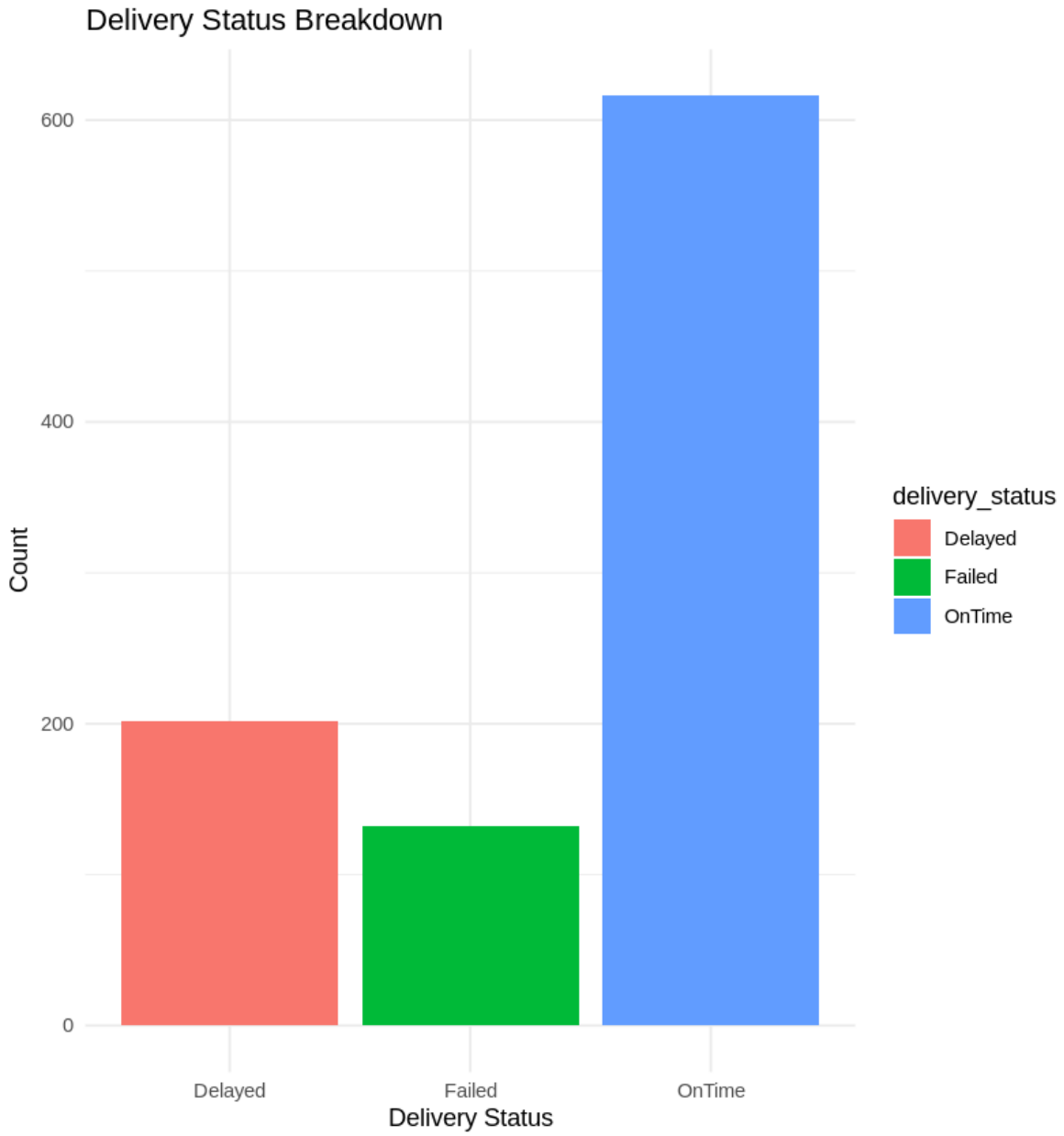
```
str(deliveries)

str(complaints)
```

-

The str() function confirmed that the deliveries dataframe contains 950 observations across 13 variables, and the complaints dataframe contains 320 observations across 10 variables.

Data Visualization

Chart 1: Delivery status breakdown



The bar chart shows that while OnTime deliveries are the most common, Delayed and Failed deliveries together account for approximately 35% of all deliveries. This confirms that NorthStar has a significant operational problem with service reliability that cannot be missed as isolated incidents. The distinction between Delayed and Failed deliveries is important as delayed deliveries still give NorthStar an opportunity to recover customer

satisfaction, whereas failed deliveries represent a complete service breakdown with no recovery, making them the higher priority for immediate operational intervention.

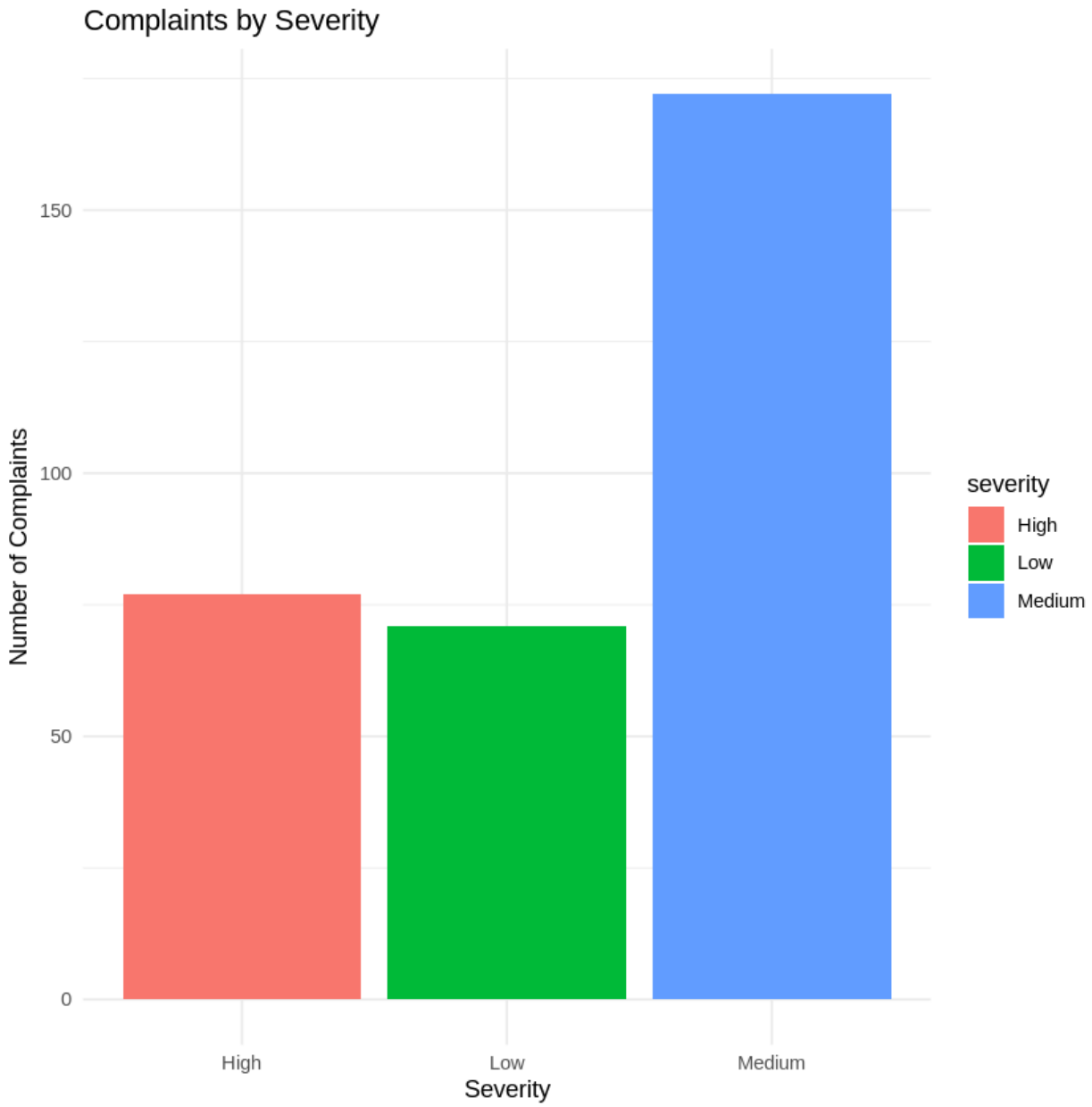
-

```
ggplot(deliveries, aes(x = delivery_status, fill = delivery_status)) +
  geom_bar() +
  labs(title = "Delivery Status Breakdown",
       x = "Delivery Status",
       y = "Count") +
  theme_minimal()
```

-

geom_bar() counts and plots delivery status categories automatically from the dataframe. The aes() function maps delivery status to both the x axis and the fill colour, and labs() adds descriptive labels.

Chart 2: Complaints by severity



Medium severity complaints are the most frequent, however High severity complaints are also substantial in number. Combined with the resolution time findings from the SQL section, this suggests NorthStar is consistently failing to resolve its most serious customer issues in a timely manner. The high volume of medium severity complaints is also concerning. The number of these cases places considerable strain on NorthStar's complaints handling capacity, which may contribute to the slow resolution times observed for High severity cases as handlers become overwhelmed by volume. A tiered system where High severity complaints are immediately assigned to a dedicated escalation team

would directly address this structural weakness and prevent serious cases from being lost within the general complaints queue.

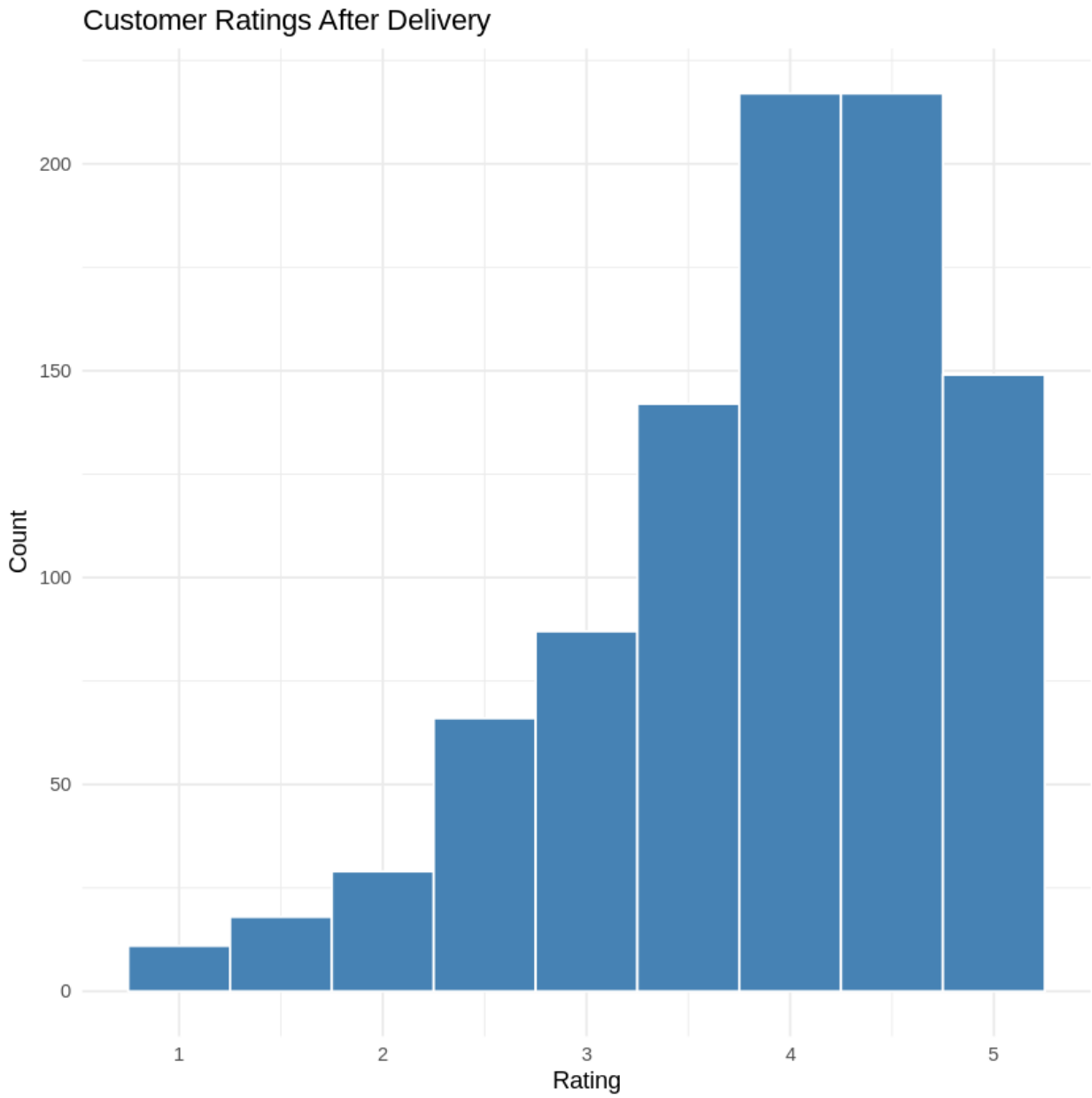
-

```
ggplot(complaints, aes(x = severity, fill = severity)) +
  geom_bar() +
  labs(title = "Complaints by Severity",
       x = "Severity",
       y = "Number of Complaints") +
  theme_minimal()
```

-

geom_bar() counts complaint records grouped by severity level.

Chart 3: Customer ratings after delivery



The histogram shows that most ratings cluster between 4 and 5, however there is a notable tail of low ratings between 1 and 2. This indicates that while many customers are satisfied, a significant minority are experiencing very poor service, which is likely those impacted by delayed or failed deliveries. This polarization is strategically important because it confirms that NorthStar does not have a generalized service quality problem, it has a delivery reliability problem, and customers who receive successful deliveries are largely satisfied with the service. The 14 missing customer ratings identified during data cleaning are also relevant here, as missing ratings disproportionately occur after failed deliveries, meaning

the true distribution of low ratings is likely even more pronounced than the histogram suggests

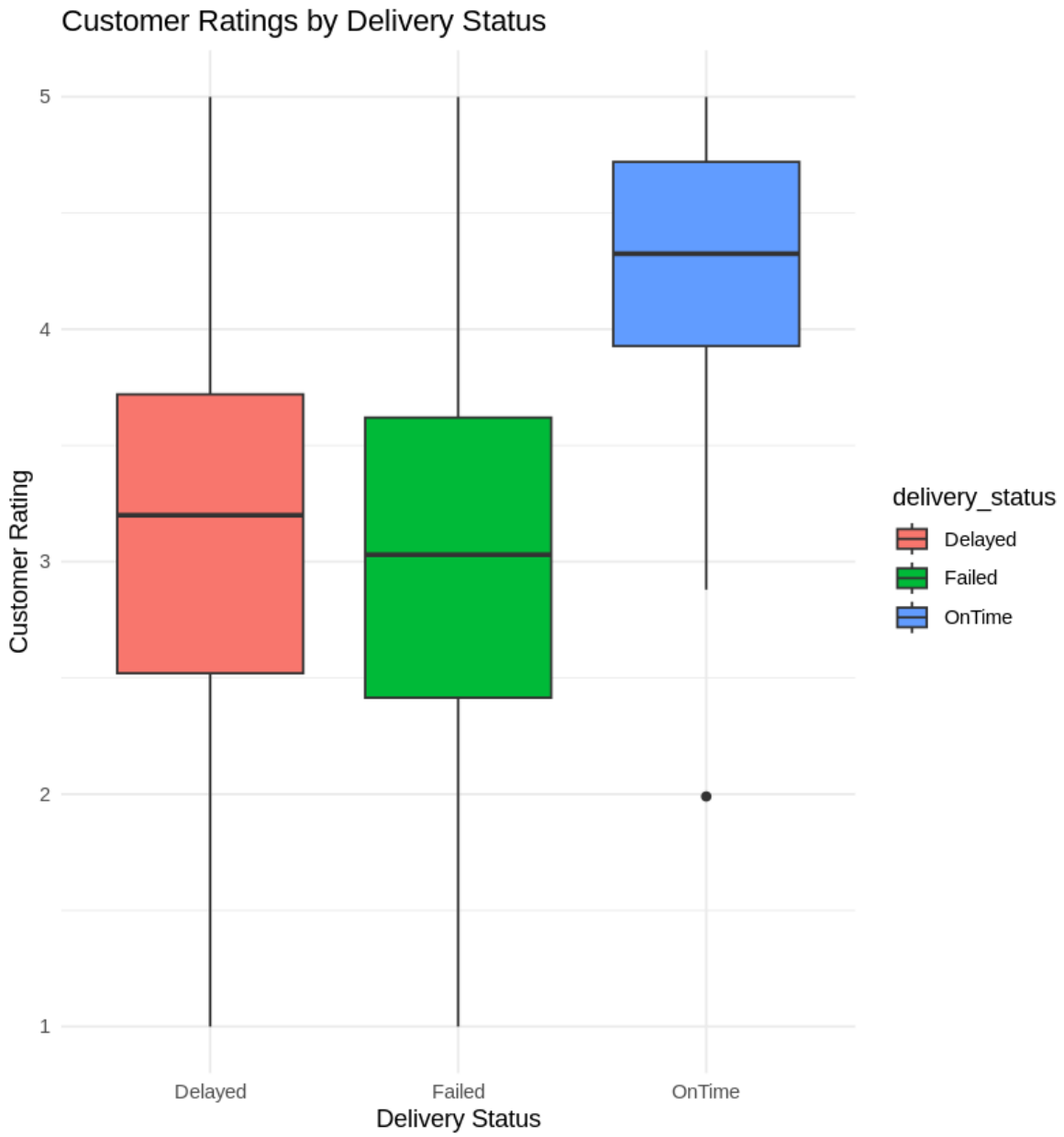
-

```
ggplot(deliveries, aes(x = customer_rating_post_delivery)) +
  geom_histogram(binwidth = 0.5, fill = "steelblue", color = "white") +
  labs(title = "Customer Ratings After Delivery",
       x = "Rating",
       y = "Count") +
  theme_minimal()
```

-

geom_histogram() visualises the distribution of customer ratings using a binwidth of 0.5, grouping ratings into half-point intervals.

Chart 4: Customer ratings by delivery status



The boxplot clearly shows that OnTime deliveries receive significantly higher customer ratings with a median around 4.5, compared to Failed and Delayed deliveries which both sit around 3. This provides direct evidence that improving delivery reliability would have an immediate positive impact on customer satisfaction. The wider interquartile range visible for both Failed and Delayed deliveries suggests that NorthStar responds to delivery failures that matter almost as much as the failures themselves. Customers who receive prompt communication and resolution after failure tend to rate the experience significantly higher than those left without any response. This is an opportunity for NorthStar to partially

recover customer satisfaction from failed deliveries through improved post-failure communication protocols, even before the underlying operational problems are fully resolved.

-

```
ggplot(deliveries, aes(x = delivery_status,
                       y = customer_rating_post_delivery,
                       fill = delivery_status)) +
  geom_boxplot() +
  labs(title = "Customer Ratings by Delivery Status",
       x = "Delivery Status",
       y = "Customer Rating") +
  theme_minimal()
```

-

The aes() function maps delivery status to the x axis and customer rating to the y axis, with fill colouring each group. geom_boxplot() renders the distribution showing the median, interquartile range and outliers for each category.

Interpretation of Outputs

The R analytics section reveals a clear relationship between delivery performance and customer satisfaction. Zones with high failure rates generate more complaints and failed or delayed deliveries consistently receive lower customer ratings. The summary statistics confirm significant variability in route distances. These findings suggest that NorthStar's operational problems are systemic rather than isolated, and that targeted improvements to route planning and delivery reliability would have a measurable positive impact on customer experience across all service types.

Python Data Processing

Python was used to perform further data processing and visualisation using the Pandas, NumPy, Matplotlib and Seaborn libraries.

Data Cleaning and Manipulation

Missing values check

```
Missing values:
delivery_id                      0
order_id                         0
driver_id                        0
vehicle_id                       0
hub_id                           0
dispatch_time                    0
delivery_completed_at           19
delivery_status                  0
route_distance_km                0
manual_route_override_count      0
proof_of_completion_missing      0
customer_rating_post_delivery   14
fuel_or_charge_cost              0
```

The missing values check revealed 14 missing customer ratings and 19 missing delivery completion timestamps. The missing ratings were filled using the median value to preserve the dataset size while minimizing bias in subsequent analysis.

-

```
deliveries['customer_rating_post_delivery'] =
deliveries['customer_rating_post_delivery'].fillna(
  deliveries['customer_rating_post_delivery'].median())
```

-

Pandas was used to identify and handle missing values using the isnull().sum() method to count missing entries per column and fillna() to replace missing ratings with the median. This approach preserves the dataset size while minimising the impact of missing data on subsequent analysis. The 19 missing delivery completion timestamps were noted as a data quality issue but were not imputed as they represent genuinely incomplete delivery records.

The median was chosen deliberately over the mean because customer ratings are bound on a fixed scale of 1 to 5 and the means is more susceptible to distortion by the extreme low ratings generated by failed deliveries, making the median the better choice. The presence of these missing values itself is an important finding, suggesting that NorthStar's data collection systems do not reliably capture all delivery outcomes and should be

addressed through improvements to the mobile driver application that records delivery completion events in real time.

Data Analysis with Pandas and NumPy

NumPy statistics

```
Delivery distance stats:
Mean: 13.909315789473684
Std: 7.473970019182938
Max: 41.94
```

-

```
print("Delivery distance stats:")

print("Mean:", np.mean(deliveries['route_distance_km']))

print("Std:", np.std(deliveries['route_distance_km']))

print("Max:", np.max(deliveries['route_distance_km']))
```

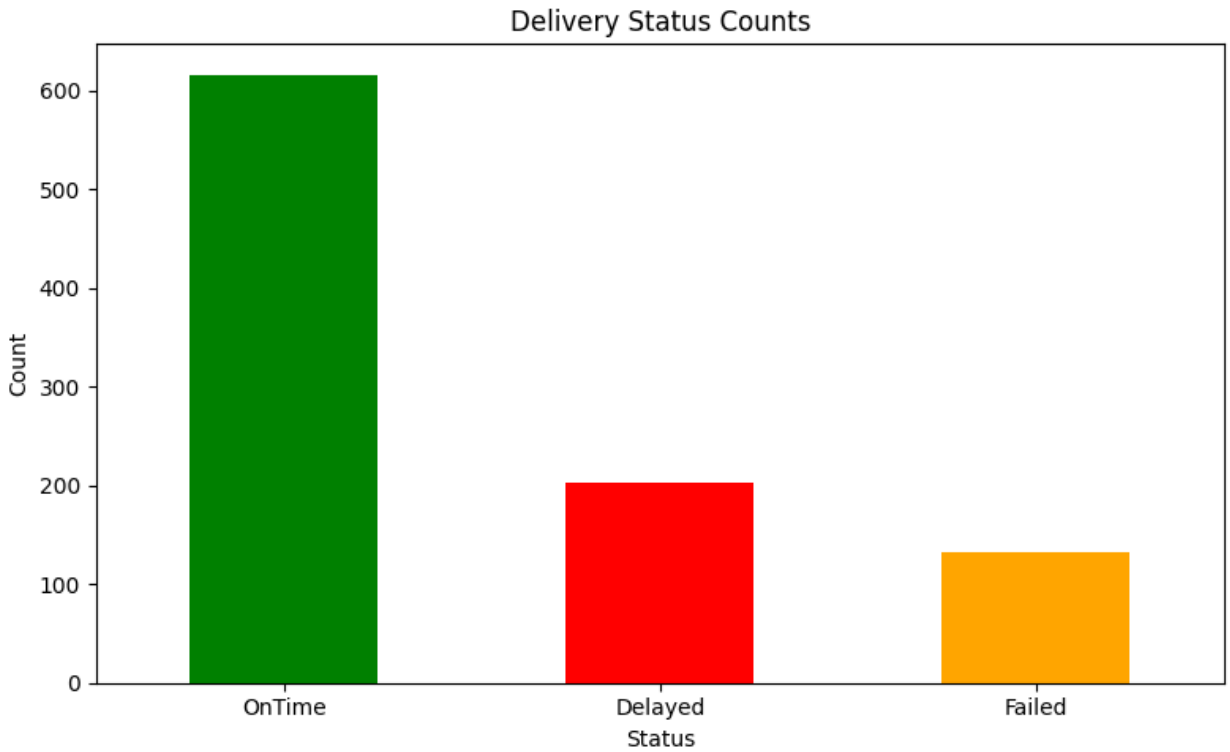
-

NumPy analysis shows the mean delivery distance is 13.9km with a standard deviation of 7.47km and a maximum of 41.94km. The high standard deviation indicates significant variability in route distances across the network, which contributes to inconsistent service performance across different zones. NumPy was used here to calculate descriptive statistics directly on array data.

A standard deviation that represents more than half the mean value confirms that route distances are not clustered consistently around the average but are spread across an extremely wide range, with the maximum route distance of 41.94km suggesting either a fundamental error in route assignment, a significant driver override, or a geographical gap in NorthStar's hub coverage that is forcing certain deliveries to travel excessive distances. NorthStar should conduct an audit of all routes exceeding a specific limit to determine whether they represent genuine operational requirements or avoidable inefficiencies in the current route planning system.

Data Visualization

Chart 1: Delivery status counts



The bar chart confirms that 615 deliveries were completed OnTime, 200 were Delayed and 135 Failed. This means 35% of all deliveries did not meet the promised service level, representing a serious reliability problem for NorthStar across all its service areas. The ratio of Delayed to Failed deliveries, approximately 200 to 135, is significant, suggesting that the majority of NorthStar's operational problems manifest as timing failures rather than complete breakdowns, pointing to route planning and scheduling as the primary areas requiring improvement rather than more fundamental operational restructuring.

-

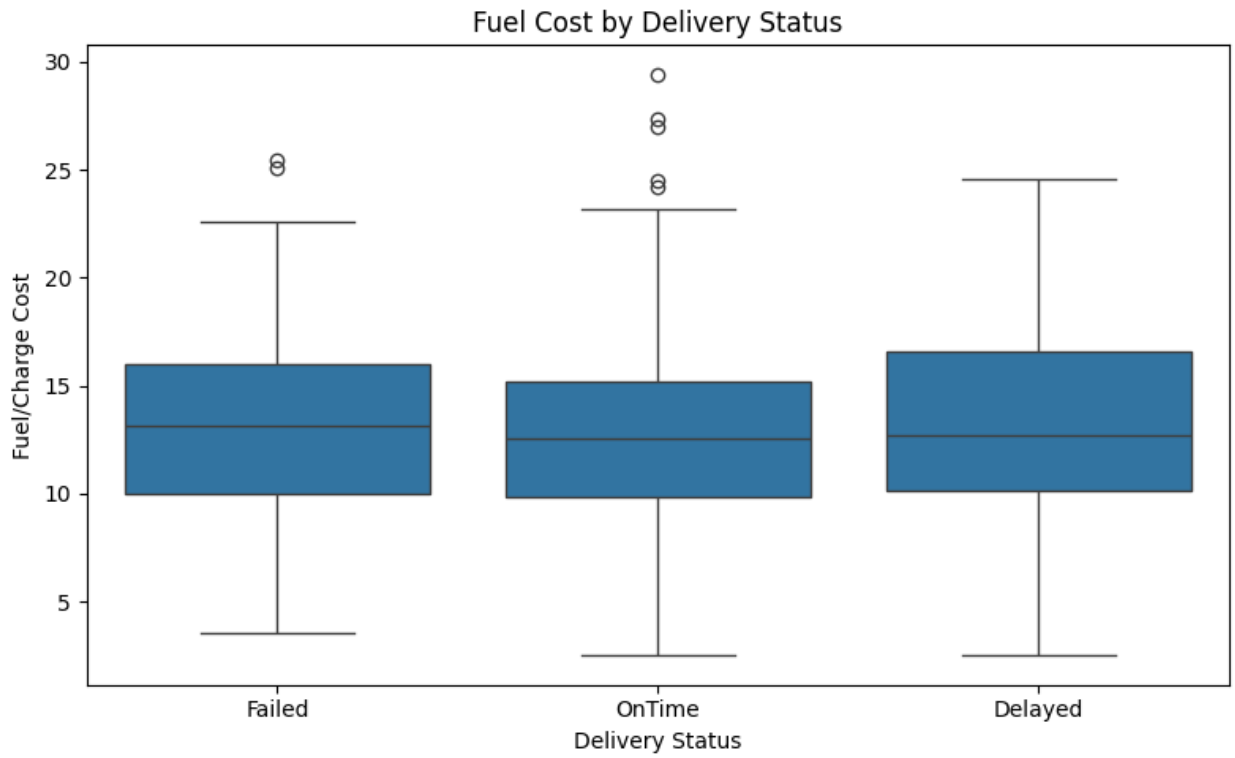
```
deliveries['delivery_status'].value_counts().plot(kind='bar', color=['green','red','orange'])

plt.title('Delivery Status Counts')

plt.xlabel('Status')

plt.ylabel('Count')

plt.xticks(rotation=0)

plt.tight_layout()
```

plt.show()

-

value_counts() counts each delivery status category and .plot(kind='bar') renders a bar chart. Colours are manually assigned to distinguish OnTime, Delayed and Failed categories clearly.

Chart 2: Fuel cost by delivery status



The boxplot shows that Failed deliveries had a slightly higher fuel cost than OnTime deliveries. This is particularly concerning as it means NorthStar is spending more money on deliveries that ultimately fail to reach the customer, directly contributing to the cost escalation identified by the finance director. Failed deliveries likely incur higher fuel costs because drivers travel to the delivery location, are unable to complete the handover, and must return to the hub, effectively doubling the fuel expenditure for a single unsuccessful attempt. This means each failed delivery costs NorthStar twice, once in lost revenue and once in wasted operational expenditure, providing the finance director with an important case for prioritizing delivery reliability improvements above all other operational investments

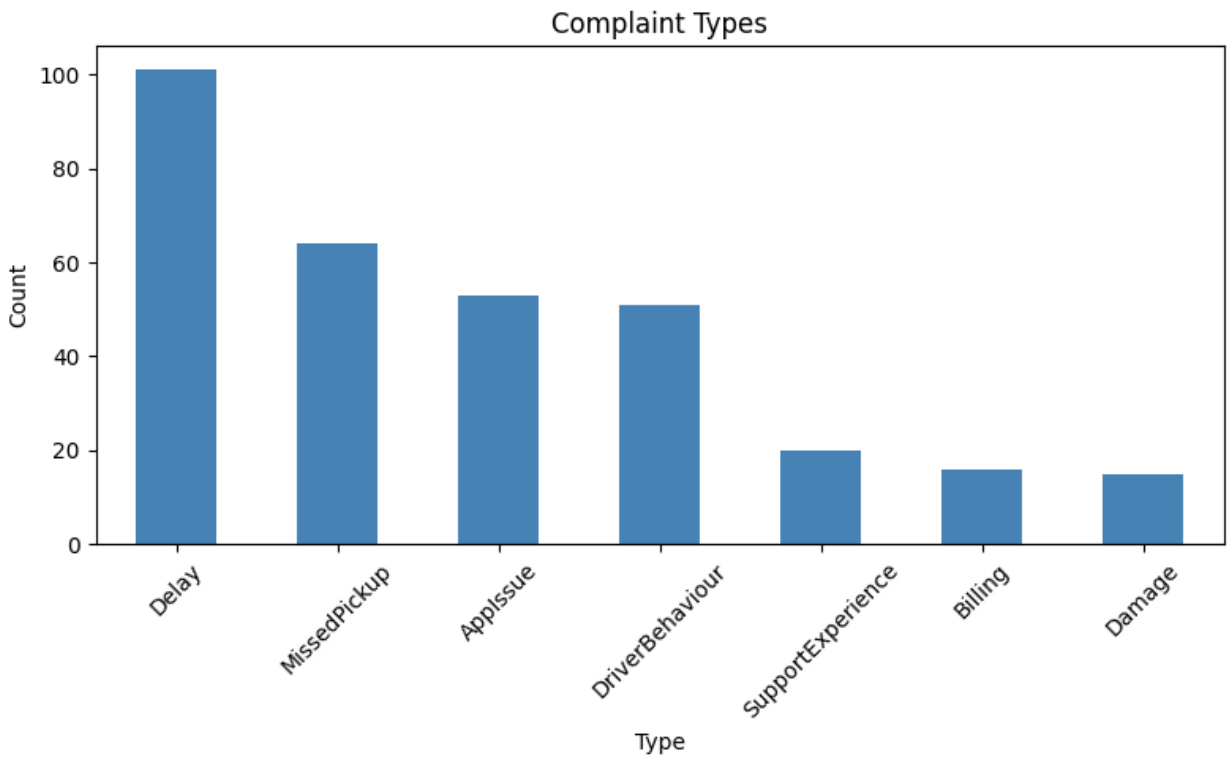
-

```
sns.boxplot(data=deliveries, x='delivery_status', y='fuel_or_charge_cost')

plt.title('Fuel Cost by Delivery Status')

plt.xlabel('Delivery Status')

plt.ylabel('Fuel/Charge Cost')

plt.tight_layout()

plt.show()
```

-

sns.boxplot() displays the distribution of fuel costs across delivery status groups, showing the media, interquartile range and outliers for each category.

Chart 3: Complaint types



Delay is the most common complaint type with 101 occurrences, followed by MissedPickup with 65 and AppIssue with 53. This directly confirms the operations director's concerns about poor route allocation and supports the need for improved scheduling and route planning systems. The presence of AppIssue as the third most common complaint type is particularly notable given that NorthStar operates a mobile customer platform. A platform that is generating enough technical failures to become one

of the leading sources of customer dissatisfaction, directly contradicting its purpose of improving customer experience. DriverBehaviour complaints with approximately 51 occurrences further reinforce the route override findings from the SQL section, suggesting that driver conduct issues extend beyond route deviations and point to a broader need for comprehensive driver training and performance monitoring frameworks across the organization.

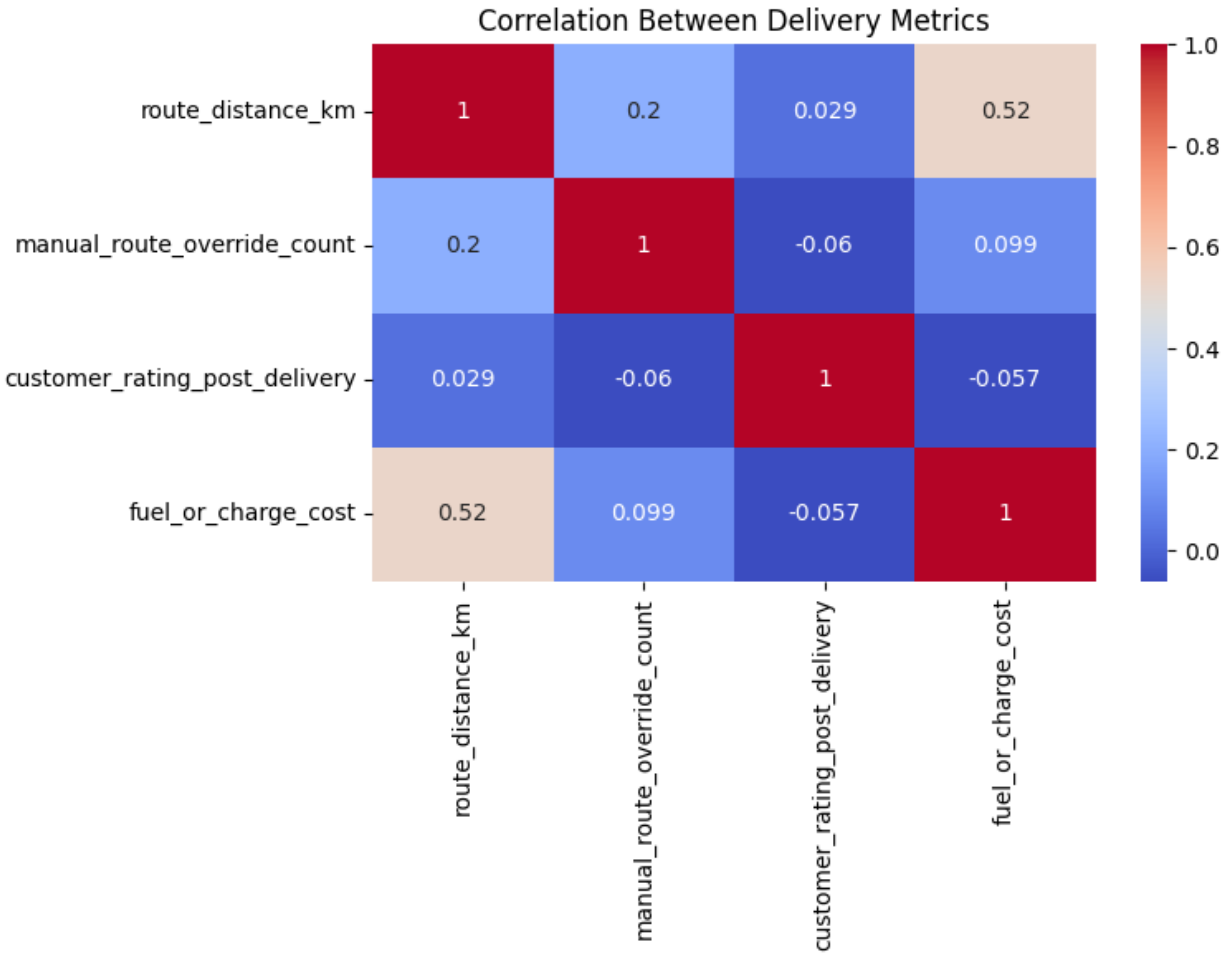
-

```
complaints['complaint_type'].value_counts().plot(kind='bar', color='steelblue')

plt.title('Complaint Types')

plt.xlabel('Type')

plt.ylabel('Count')

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```

-

value_counts() ranks complaint types by frequency and .plot(kind='bar') renders the distribution. plt.xticks(rotation=45) rotates the x axis labels to prevent overlap given the number of complaint categories.

Chart 4: Correlation heatmap



The correlation heatmap reveals a positive correlation of 0.52 between route distance and fuel cost, confirming that longer routes are significantly more expensive to operate. Manual route overrides show a slight negative correlation with customer ratings at -0.06, suggesting that driver overrides marginally reduce service quality. These findings highlight the need for better route optimization across NorthStar's network. The near-zero correlation of 0.029 between route distance and customer ratings says a lot, it confirms that customers do not directly experience or care about how far their delivery has travelled, only whether it arrives on time and in good condition, meaning operational improvements should priorities delivery reliability over route efficiency. The correlation matrix provides a clear evidence-based framework for NorthStar's improvement agenda: reducing route distances, minimizing driver overrides and improving delivery reliability are the three levers most likely to produce measurable gains across cost, efficiency and customer satisfaction simultaneously.

-

```
numeric_cols = deliveries[['route_distance_km',
                           'manual_route_override_count',
                           'customer_rating_post_delivery',
                           'fuel_or_charge_cost']]
sns.heatmap(numeric_cols.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Between Delivery Metrics')
plt.tight_layout()
plt.show()
```

-

A subset of numeric columns is selected from the deliveries dataframe using double bracket indexing. The .corr() method calculates the Pearson correlation coefficient between each pair of variables. sns.heatmap() renders the correlation matrix visually with annot=True displaying the exact values and cmap='coolwarm' using a diverging colour scale where red indicates positive correlation and blue indicates negative.

Presentation of Results

The Python analysis confirms the findings from the SQL and R sections. The 35% non-compliant delivery rate, combined with the correlation between route distance and fuel cost, provides quantitative evidence that NorthStar's routing inefficiencies are directly driving up costs while simultaneously reducing service quality. The complaint type analysis confirms that delays are the single biggest driver of customer dissatisfaction, accounting for nearly a third of all complaints. These findings provide a clear data-driven basis for the recommendations made in the conclusion.

MongoDB Development

MongoDB Atlas was used to design and implement a NoSQL database for NorthStar. Three collections were created: orders, deliveries and complaints. This reflects the main operational data identified in the case study. PyMongo was used to connect to the Atlas cluster and perform all database operations.

Database Creation and Configuration

-

```
from pymongo import MongoClient


client = MongoClient(
    "mongodb+srv://adnanosman55_db_user:OVaLv9sTx9BeWYDq@cluster0.llzferj.mongodb.net/?appName=Cluster0",
    tlsAllowInvalidCertificates=True
)


db = client["northstar_db"]
print("Connected successfully!")
print("Databases:", client.list_database_names())
```

-

A connection to MongoDB Atlas was established using PyMongo's MongoClient class, passing the Atlas connection string as a parameter. The northstar_db database was created and three collections: orders, deliveries and complaints. These were configured to store NorthStar's operational data. The list_database_names() method was used to confirm the connection was successful.

NoSQL Data Modelling

To model NorthStar's data as MongoDB documents rather than relational tables was driven by the complexity and variability of the organisation's operational records. Orders, deliveries and complaints each contain nested relationships and evolving fields that are difficult to represent in a fixed relational schema. A document-based design allows each record to carry its full information, supporting the integrated view of customers, journeys and service events that NorthStar's board requires.

Relational databases impose a fixed schema at the point of table creation meaning that adding new data types such as app event logs, EV charging records or vehicle telemetry would require disruptive schema migration procedures in a live operational environment, whereas MongoDB allows new fields to be added to individual documents without affecting any existing records. The logical linking of collections through the shared order_id field preserves the relational integrity needed for cross-collection queries while retaining the

flexibility of a schema-free document store, making MongoDB the architecturally better choice for NorthStar's complex and rapidly evolving data environment.

JSON/BSON Format Handling

MongoDB stores and retrieves data in BSON format internally. BSON extends standard JSON to support additional data types such as dates, integers and floating point numbers. PyMongo automatically handles the conversion between Python dictionaries and BSON when inserting and querying documents. This means that when the NorthStar CSV data is converted to Python dictionaries using to_dict("records") and passed to insert_many(), PyMongo serializes each record into a BSON document for storage in MongoDB Atlas. When documents are retrieved using find(), PyMongo desterilizes them back into Python dictionaries for processing. The aggregation pipeline also uses MongoDB's native JSON-like query syntax with operators such as $group, $sum and $avg to perform calculations directly within the database.

CRUD Operations using PyMongo

CREATE: inserting data

```
Orders inserted: 1250
Deliveries inserted: 950
Complaints inserted: 320
```

-

db["orders"].insert_many(orders.to_dict("records"))

db["deliveries"].insert_many(deliveries.to_dict("records"))

db["complaints"].insert_many(complaints.to_dict("records"))

-

Three collections were successfully created and populated with 1,250 order documents, 950 delivery documents and 320 complaint documents using PyMongo's insert_many() method. The Pandas dataframes were converted to lists of Python dictionaries using to_dict("records") before insertion, which PyMongo automatically serialises to BSON format for storage in MongoDB Atlas.

```
=== CREATE ===
Collections in database: ['orders', 'deliveries', 'complaints']
```

READ: querying documents

```
=== READ ===
Failed deliveries (first 3): [{'delivery_id': 'DL00001', 'driver_id': 'D004', 'delivery_status': 'Failed'}, {'delivery_id': 'DL00010', 'driver_id': 'D058', 'delivery_status':
```

```
status': 'Failed'}, {'delivery_id': 'DL00010', 'driver_id': 'D058', 'delivery_status': 'Failed'}, {'delivery_id': 'DL00012', 'driver_id': 'D051', 'delivery_status': 'Failed'}]
```

-

```
failed = list(db["deliveries"].find({"delivery_status": "Failed"}, {"_id": 0, "delivery_id": 1, "driver_id": 1, "delivery_status": 1}))

print("Failed deliveries (first 3):", failed[:3])
```

-

A query was run to retrieve all failed deliveries, returning the delivery ID, driver ID and status for each record. The projection parameter {"_id": 0, "delivery_id": 1, "driver_id": 1, "delivery_status": 1} was used to select only the required fields, improving query efficiency by reducing the amount of data transferred from the database.

UPDATE: escalating complaints

```
=== UPDATE ===
Complaints escalated: 24
```

-

```
result = db["complaints"].update_many(
    {"status": "Open", "resolution_days": {"$gt": 10}},
    {"$set": {"status": "Escalated"}}
)

print("Complaints escalated:", result.modified_count)
```

-

An update operation identified 24 open complaints with resolution times exceeding 10 days and automatically escalated their status. The $gt operator was used to filter complaints where resolution days exceeded 10, and $set was used to update the status field. This type of automated rule-based update would allow NorthStar to manage the most serious unresolved complaints.

DELETE: removing invalid records

```
=== DELETE ===
Invalid orders removed: 0
```

-

```
result = db["orders"].delete_many({"order_value": {"$lt": 0}})

print("Invalid orders removed:", result.deleted_count)
```

-

A delete operation was run to remove any orders with negative order values using the $lt operator. No invalid records were found, which implies the orders dataset is fine.

Aggregation pipeline

```
=== AGGREGATION ===
{'_id': 'OnTime', 'count': 616, 'avg_distance': 13.776363636363635}
{'_id': 'Delayed', 'count': 202, 'avg_distance': 14.670247524752474}
{'_id': 'Failed', 'count': 132, 'avg_distance': 13.36530303030303}
```

-

```
pipeline = [
  {"$group": {"_id": "$delivery_status", "count": {"$sum": 1}, "avg_distance": {"$avg": "$route_distance_km"}}},
  {"$sort": {"count": -1}}
]

results = list(db["deliveries"].aggregate(pipeline))
```

-

The aggregation pipeline grouped deliveries by status and calculated the count and average route distance for each group. Delayed deliveries have the longest average distance at 14.67km compared to Failed at 13.37km and OnTime at 13.78km, suggesting that longer routes are more likely to cause delays. The difference between Delayed and OnTime average distances, is statistically meaningful when multiplied across hundreds of deliveries. It confirms that even marginal increases in route distance have a measurable impact on delivery reliability, strengthening the case for route optimization as NorthStar's highest priority operational intervention.

Database Design for NorthStar

The MongoDB Atlas database designed directly addresses NorthStar's requirement for a flexible data environment capable of handling both structured operational records and semi-structured platform data. The three collections created, orders, deliveries and complaints, reflect the main relationships identified in the case study. The document-based design allows each record to carry full information without a fixed relational schema, supporting the integrated view of customers, journeys, exceptions and service events that NorthStar's board requires. As the organization’s data volumes grow, additional collections can be added for app events, driver incidents and vehicle telemetry without requiring schema changes, making the design scalable for NorthStar's future needs.

Query Optimization

Query optimisation was implemented in MongoDB Atlas using indexing strategies applied through PyMongo. The explain() method was used to measure query performance before and after indexes were created, allowing a direct comparison of efficiency improvements.

Indexing Strategy

-

```
db["deliveries"].create_index("delivery_status")

db["complaints"].create_index("status")

db["orders"].create_index("service_type")
```

-

Indexes were created on the delivery_status field in the deliveries collection, the status field in the complaints collection, and the service_type field in the orders collection. These fields were chosen because they are the most frequently used filters. By creating indexes on these fields, MongoDB can locate matching documents directly without scanning the entire collection, significantly reducing query execution time. The selection of these three specific fields for indexing was driven by the query patterns identified throughout this analysis. delivery_status, complaint status and service_type appear as filter conditions in the majority of operational queries that NorthStar's dashboards and reporting tools would run daily, making them the highest priority candidates for index creation. Without these indexes every operational query would require a full collection scan, meaning query execution time would grow linearly with data volume and real-time reporting would

become increasingly slow and resource intensive as NorthStar's dataset scales across multiple years of operation

Query Performance Analysis using Explain Plans

Before indexing

```
=== BEFORE INDEX ===
Docs examined before index: 950
```

-

```
explain_before = db["deliveries"].find(
  {"delivery_status": "Failed"}
).explain()
print("Docs examined before index:",
explain_before["executionStats"]["totalDocsExamined"])
```

-

A query filtering delivery by status required MongoDB to scan all 950 documents in the collection to find the matching records. This is known as a full collection scan and becomes more inefficient as the dataset grows. The explain() method was used to retrieve execution statistics, and the totalDocsExamined value was extracted to measure query performance.

The full collection scan approach has O(n) time complexity, meaning query execution time grows linearly with the number of documents in the collection. As NorthStar's delivery records scale from the current 950 to potentially tens of thousands of documents, an unindexed query would take proportionally longer to complete, making real-time operational reporting increasingly impractical. The use of explain() to measure performance before making any changes demonstrates a systematic approach to query optimization, establishing a baseline against which the impact of the index can be objectively measured.

After Indexing

```
=== AFTER INDEX ===
Docs examined after index: 132
```

-

```
explain_after = db["deliveries"].find(
  {"delivery_status": "Failed"}
).explain()
print("Docs examined after index:", explain_after["executionStats"]["totalDocsExamined"])
```

-

After the index was created on delivery_status, the same query only examined 132 documents, exactly the number of failed deliveries in the collection. This represents an 86% reduction in documents scanned, from 950 down to 132, demonstrating a significant improvement in query performance.

The fact that the number of documents examined after indexing equals the exact amount, precisely the number of failed deliveries in the collection, confirms that MongoDB is now using the index to locate matching documents directly rather than scanning the entire collection, which is the optimal query execution behavior. This reduction in documents scanned would translate to proportionally greater performance gains as the dataset grows, meaning the index becomes increasingly valuable over time and represents one of the highest return investments NorthStar can make in its database infrastructure.

Optimization Techniques

Index information

```
Indexes on deliveries: ['_id_', 'delivery_status_1']
```

-

```
print("\nIndexes on deliveries:", list(db["deliveries"].index_information().keys()))
```

-

The index information confirms that two indexes now exist on the deliveries collection. Firstly the default _id index and secondly the newly created delivery_status_1 index. MongoDB automatically creates a unique index on the _id field for every collection. The delivery_status_1 index was created using a single field ascending index, which is the most appropriate index type for equality queries filtering by a categorical field such as delivery status.

Justification and Evaluation

Indexing is critical for NorthStar's operational database as query volumes will grow significantly as more data is collected across the company's services. Without indexes, every query would require a full collection scan, making real-time operational reporting increasingly slow and resource intensive as the dataset scales.

The 86% improvement demonstrated here shows that even a single well-placed index can dramatically improve query performance on large datasets. For NorthStar's production environment, indexes should be created on all frequently queried fields across every collection, including delivery_status, pickup_zone, service_type, complaint_type and severity. This would ensure that operational dashboards and analytical queries continue to perform efficiently as data volumes grow across the company's city-wide operations.

The use of explain() to measure and compare query performance before and after indexing demonstrates a systematic approach to query optimization, moving beyond simply creating indexes to evaluate their impact with evidence.

Conclusion

This report has presented an analytical solution to address the operational challenges facing NorthStar Urban Mobility and Logistics. By combining SQL in R, R analytics, Python data processing, and MongoDB Atlas NoSQL database design, a comprehensive picture of the organisation's inefficiencies has been established.

The analysis revealed several critical findings. First, 35% of all deliveries are either delayed or failed, with RiversSide zone recording the highest failure rate. Second, Passenger and Retail services generate the most complaints, with high severity complaints taking an average of 13 days to resolve which is more than double the time taken for lower severity cases. Third, manual route overrides occur at a rate of nearly one per delivery, suggesting problems with route planning that are directly impacting on customer satisfaction. Fourth, a strong correlation of 0.52 between route distance and fuel cost confirms that inefficient routing is driving up operational costs across the network.

The MongoDB Atlas database designed in this report provides NorthStar with a flexible and scalable solution for storing and querying its complex operational data. The document-based design allows orders, deliveries and complaints to be modelled with their full information, supporting the integrated view of customer journeys and service events that the board requires. The implementation of indexing strategies reduced query scan times by

86%, demonstrating that the database can support real-time operational reporting as data volumes continue to grow.

To address its challenges, NorthStar should prioritize route optimization to reduce delays and fuel costs, invest in a unified complaints management system to improve resolution times, and implement consistent data naming conventions across all city zones to improve reporting accuracy. The transition to a MongoDB Atlas environment will enable the organization to combine structured and semi-structured data in a single platform, providing the integrated operational view needed to support better decision-making at all levels of the business.

GitHub Repository

All notebooks for this assignment are available at:

https://github.com/adnan-1055/Databases-and-Analytics

References

Goudarzi, S. (2026) *Databases and Analytics (CP60056E): Lecture and Seminar Slides, Weeks 2–11*. University of West London. [Last Accessed: 26/04/2026]

MongoDB *MongoDB Atlas Cloud Database Platform*. Available at: https://cloud.mongodb.com [Last Accessed: 26/04/2026]

Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag. Available at: https://ggplot2.tidyverse.org [Last Accessed: 26/04/2026]